

can be too memory intensive

torch.no_grad()

f.detach()



autodiff all over natural compute/accuracy tradeoff


```
# Use envelope theorem differentiation for memory saving  
res = ot.solve(M, a, b, reg=1.0, grad='envelope') # M, a, b are torch tensors  
res.value.backward() # only the value is differentiable
```

Note that by default the Sinkhorn solver uses automatic differentiation to compute the gradients of the values and plan. This can be changed with the *grad* parameter. The *envelope* mode computes the gradients only for the value and the other outputs are detached. This is useful for memory saving when only the gradient of value is needed.

```
ot.solve(M, a=None, b=None, reg=None, c=None, reg_type='KL',  
unbalanced=None, unbalanced_type='KL', method=None, n_threads=1,  
max_iter=None, plan_init=None, potentials_init=None, tol=None,  
verbose=False, grad='autodiff') \[source\]
```

Solve the discrete optimal transport problem and return

`OTResult` object

- **grad** (*str*, optional) – Type of gradient computation, either or 'autodiff', 'envelope' or 'last_step' used only for Sinkhorn solver. By default 'autodiff' provides gradients wrt all outputs (*plan*, *value*, *value_linear*) but with important memory cost. 'envelope' provides gradients only for *value* and and other outputs are detached. This is useful for memory saving when only the value is needed. 'last_step' provides gradients only for the last iteration of the Sinkhorn solver, but provides gradient for both the OT plan and the objective values. 'detach' does not compute the gradients for the Sinkhorn solver.

Stop Grad Trick

- Back-propagating through all Sinkhorn iters **can be too memory intensive** (because it stores intermediate activations)
- But at convergence, we know: $\nabla_{\mathbf{a}} \text{OT}_c^\epsilon(\mathbf{a}, \mathbf{b}) = \mathbf{f} \Rightarrow$ can use the last iterate $\mathbf{f}^{(L)}$ as surrogate for the gradient, stop backprop from flowing into Sinkhorn.
- To stop gradient tracking and avoid storing activations:
 - Option 1: Run Sinkhorn inside **`torch.no_grad()`**
 - Option 2: Retroactively cut computation graph using **`f.detach()`**
- Takeaway: **autodiff allows natural compute/accuracy tradeoff**

```
ot.solve(M, a=None, b=None, reg=None, c=None, reg_type='KL',  
unbalanced=None, unbalanced_type='KL', method=None, n_threads=1,  
max_iter=None, plan_init=None, potentials_init=None, tol=None,  
verbose=False, grad='autodiff') \[source\]
```

Solve the discrete optimal transport problem and return

`OTResult` object

- **grad** (*str, optional*) – Type of gradient computation, either or 'autodiff', 'envelope' or 'last_step' used only for Sinkhorn solver. By default 'autodiff' provides gradients wrt all outputs (*plan*, *value*, *value_linear*) but with important memory cost. 'envelope' provides gradients only for *value* and other outputs are detached. This is useful for memory saving when only the value is needed. 'last_step' provides gradients only for the last iteration of the Sinkhorn solver, but provides gradient for both the OT plan and the objective values. 'detach' does not compute the gradients for the Sinkhorn solver.

Mini-Batch Estimation